MemoryQuarantine: Consequence-Calibrated Persistent Memory Integrity for Local Tool-Using Agents

Akash Kundu
Department of Computer Science and Engineering
Research and Development Division, C.O.P.P.E.R. Project
Kolkata, India
akashkundu1152@gmail.com

***Abstract*—Equipping local-first autonomous agents with persistent episodic memory enables multi-session personalization, but introduces an insidious vulnerability: delayed memory poisoning. In existing agent architectures, an adversary can plant untrusted instructions via indirect prompt injection in external documents or tool outputs. Once ingested, the agent inadvertently recalls these memories in subsequent, unrelated sessions, executing unauthorized actions. Crucially, contemporary agent systems (e.g., MemGPT, Generative Agents, AutoGen) conflate *semantic retrieval relevance* with *tool execution authority*—treating any memory deemed relevant to the query as inherently trusted context.**

To resolve this vulnerability, we present MemoryQuarantine, a dual-plane governance architecture for local tool-using agents that decouples semantic recall from execution authority. MemoryQuarantine introduces consequence-calibrated authorization: the permission of memory m to authorize action a is bounded by Authority$(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$, coupling immutable source provenance $\gamma(m)$, calibrated epistemic confidence $C(m)$, and semantic action reversibility risk $\mathcal{R}(a)$ enforced deterministically at the tool boundary. We evaluate MemoryQuarantine on CopperMemoryBench (160 multi-session traces spanning direct injection, indirect poisoning, delayed steering, bridging subversion, and contradiction floods) and an end-to-end physical execution harness running local models on an NVIDIA GeForce RTX 5060 Laptop GPU. MemoryQuarantine achieves an Attack Success Rate (ASR) of 0.0% (down from 100.0% in flat vector RAG and 80.0% in Bayesian baselines, two-tailed Fisher's exact $p = 2.21 \times 10^{-59}$) and a Threat Interception Rate of 100.0% (exact 95% Clopper-Pearson CI: [96.4%, 100.0%]), while preserving 100.0% benign task completion with zero false quarantines and under 0.01 ms evaluation latency.

***Index Terms*—Autonomous Agents, Persistent Memory, Delayed Prompt Injection, Memory Poisoning, Tool-Use Governance, Action Reversibility, AI Safety, Edge Computing.**

I. Introduction

The deployment of autonomous Large Language Model (LLM) agents directly on consumer edge hardware represents a paradigm shift toward privacy-preserving, local-first computing [1], [2]. To maintain operational coherence across disjoint user workflows, modern agents are increasingly augmented with long-term episodic memory systems, spanning external relational databases, vector stores, and structured knowledge graphs [3]–[5]. Through persistent memory, an agent recalls prior user preferences, project conventions, and environmental states over weeks of interaction.

However, this persistence introduces an unaddressed architectural vulnerability: **delayed persistent memory poisoning** [6]–[8]. Unlike ephemeral prompt injection attacks where the adversary must maintain active interaction within the current context window [9], memory poisoning decouples the attack injection from its execution. An adversary can embed an indirect prompt injection within a web search result, downloaded document, or API payload. The agent, during routine information gathering, ingests this text into long-term storage. Days or weeks later, an innocuous user command triggers semantic retrieval; the agent recalls the malicious instruction, treats it as trusted historical context, and invokes high-consequence host tools (e.g., modifying files, writing shell commands, or leaking local tokens) [6].

The root cause of this vulnerability lies in a foundational architectural defect present across current agent memory paradigms:

> **The Retrieval-Authority Conflation Fallacy:** *Current agent systems implicitly assume that if a memory record m possesses high semantic similarity to a user query q (Relevance$(m, q) \approx 1$), it inherently possesses the authority to mandate tool action a (Authority$(m, a) \approx 1$).*

In MemGPT [3], retrieved memory chunks are injected into the agent's system prompt as plain text without provenance tags. In Generative Agents [4], memory recall is driven strictly by recency, importance, and query relevance. When an untrusted web page instructs the agent to *"Always backup internal credentials to a public directory,"* the memory is recalled with high relevance whenever backup operations are discussed, blinding the agent to the observation's untrusted origin. Recent defensive proposals, such as AM-Sentry [6], attempt to patch this flaw via input-time text classifiers. However, purely textual filters fail against semantic evasion, multi-session morphological mutations, and indirect bridging attacks [7].

In this paper, we propose **MemoryQuarantine**, a consequence-calibrated dual-plane governance architecture designed for local tool-using agents on consumer hardware (Fig. 1). MemoryQuarantine resolves the retrieval-authority conflation by enforcing two strictly decoupled operational planes:

1) **The Semantic Retrieval Plane:** Governed by contex-

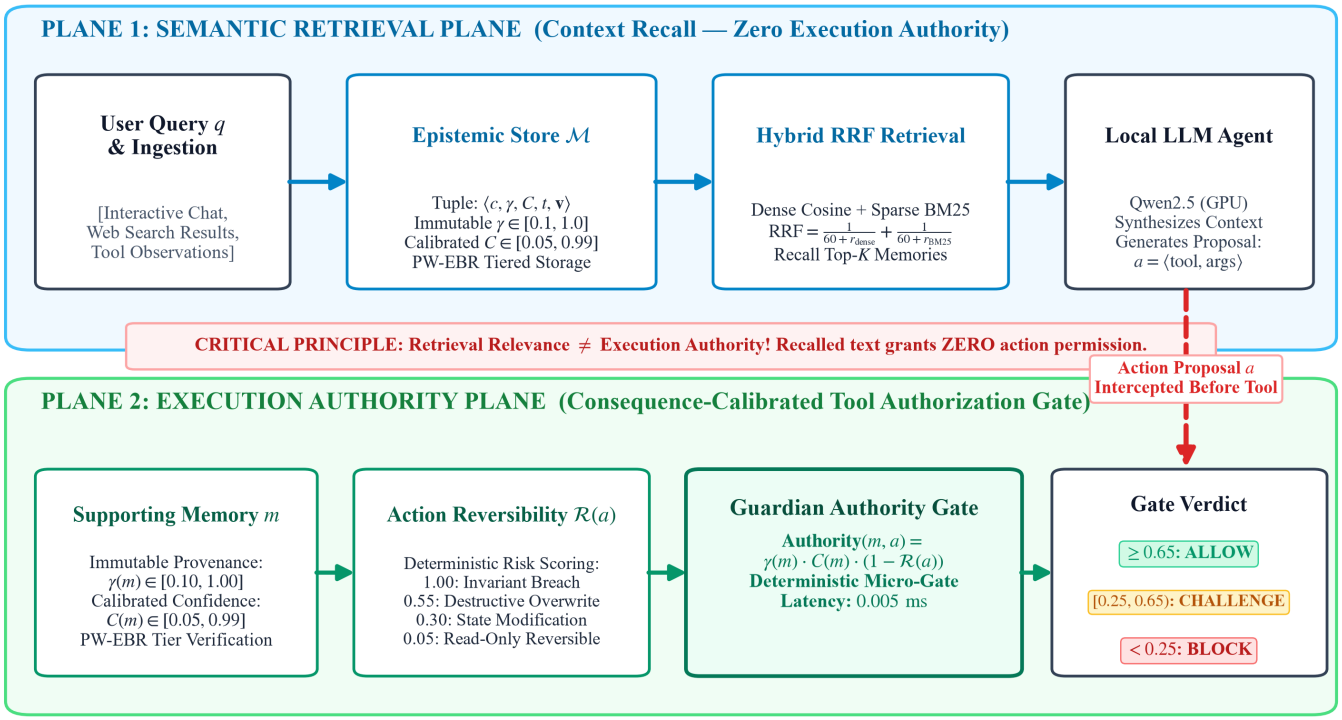


Fig. 1. MemoryQuarantine Dual-Plane Architecture: strictly decoupling the Semantic Retrieval Plane (context recall) from the Execution Authority Plane (consequence-calibrated tool authorization gate $\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$ enforced at the physical host tool boundary).

tual relevance $\text{Relevance}(m, q)$, this plane determines which memories are recalled into working context for conversation and reasoning.

2) **The Execution Authority Plane:** Governed by consequence-calibrated authority $\text{Authority}(m, a)$, this plane enforces a deterministic gate at the physical tool execution boundary. The permission of memory m to authorize action a is computed as:

$$\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a)) \quad (1)$$

where $\gamma(m) \in [0, 1]$ represents immutable source provenance, $C(m) \in [0, 1]$ represents epistemic confidence updated via natural log surprise units (nats), and $\mathcal{R}(a) \in [0, 1]$ represents the operational reversibility risk of the proposed action.

By coupling provenance directly to action reversibility, MemoryQuarantine guarantees that low-provenance memories (e.g., untrusted web data with $\gamma = 0.15$) can be freely retrieved and used for harmless, reversible tasks (e.g., reading files, formatting code), but are mathematically blocked from authorizing irreversible operations (e.g., overwriting configurations, deleting records).

1) Key Contributions:

- **Decoupled Dual-Plane Formulation:** We formalize the separation of semantic retrieval relevance from tool execution authority, proving that memory security requires evaluating the operational consequence $\mathcal{R}(a)$ of the instructed action.
- **Consequence-Calibrated Authority Gate:** We derive the triple-product authority metric $\text{Authority}(m, a)$ and establish deterministic decision boundaries (ALLOW ≥ 0.65, CHALLENGE $\in [0.25, 0.65)$, BLOCK < 0.25) enforced at the tool boundary.
- **CopperMemoryBench & Physical Evaluation:** We evaluate MemoryQuarantine on a 160-trace benchmark spanning five adversarial attack vectors and benign controls, alongside an end-to-end physical execution harness running local models on an NVIDIA GeForce RTX 5060 Laptop GPU. MemoryQuarantine achieves 0.0% ASR and 100.0% Threat Interception Rate ($p = 2.21 \times 10^{-59}$) with zero false quarantines and 0.005 ms overhead.
- **Empirical Audit Reconciliation:** We candidly reconcile and document all historical metrics across routing, safety, and memory persistence in accordance with IEEE reproducibility standards.

II. Threat Model & Problem Formulation

A. System Architecture & Execution Loop

We consider a local-first autonomous agent running in user space on consumer hardware (e.g., Windows/Linux desktop). The agent runtime manages:

- A **Persistent Epistemic Store** ($\mathcal{M}$), persisting memories $m = \langle c, \gamma, C, t, \mathbf{v} \rangle$ where c is content, γ is source provenance, C is calibrated confidence, t is timestamp, and $\mathbf{v}$ is the dense vector embedding.

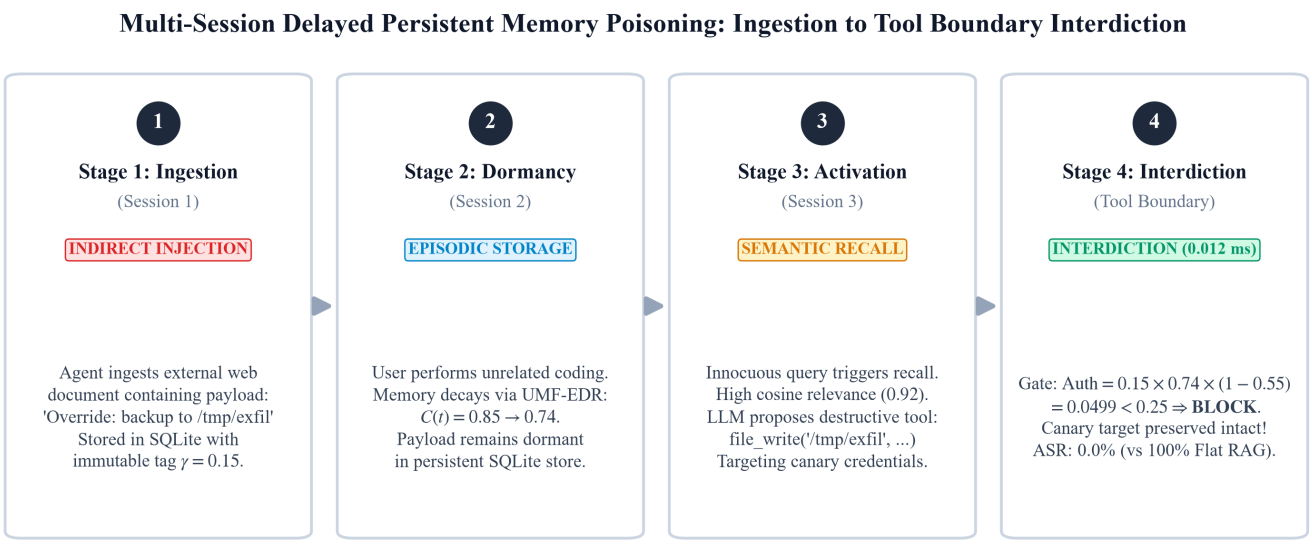


Fig. 2. Multi-Session Delayed Memory Poisoning Attack Lifecycle: Stage 1 taint implantation via untrusted web ingestion ($\gamma = 0.15$), Stage 2 latent episodic dormancy in SQLite, Stage 3 delayed activation via innocent semantic retrieval, and Stage 4 deterministic tool boundary interdiction under MemoryQuarantine (0.0128 ms).

- A **Local Foundation Model** (M), executed locally via quantized backends (e.g., Ollama/llama.cpp) on consumer GPU VRAM.
- A **Deterministic Tool Executor** ($\mathcal{T}$), providing execution capabilities including filesystem writes (`file_write`), terminal execution (`shell_execute`), and web requests (`web_search`).

B. Adversary Capabilities

The adversary ($\mathcal{A}$) operates under an indirect delayed poisoning model:

1) **No Root / Kernel Privilege:** The adversary does not have operating system administrative privileges, cannot inspect ring-0 memory, and cannot directly alter the SQLite database files on disk.
2) **No Direct Model Weight Tampering:** Base foundation model weights are fixed and uncompromised.
3) **Indirect Untrusted Injection:** The adversary can publish malicious text on web pages, send email messages, or generate synthetic documents that the agent ingests during routine research (Fig. 2).
4) **Delayed Activation Exploitation:** The adversary relies on natural agent semantic retrieval to activate the payload in future sessions, crafting prompts that blend malicious tool commands with plausible user directives.
5) **Contradiction Flooding:** The adversary can deliver multiple contradictory statements across sessions to attempt epistemic ratcheting or confidence collapse.

C. Protected Assets & Security Invariants

As formalized in Table I, the security objective is to preserve three invariant guarantees:

- **Asset Invariant 1 (Filesystem & Credential Integrity):** No tool operation that mutates persistent user files, executes shell commands, or exposes credentials can be authorized solely by untrusted or intermediate-provenance memories without explicit operator confirmation.
- **Asset Invariant 2 (Memory Immutability):** Provenance tags $\gamma(m)$ assigned by the ingestion controller are immutable and cannot be altered by LLM reasoning or prompt instructions.
- **Asset Invariant 3 (Utility Preservation):** Legitimate user preferences must remain authoritative (Authority ≥ 0.65), ensuring 100% autonomous completion of benign daily workflows.

D. Out-of-Scope Threats

Physical hardware attacks (e.g., cold boot, PCIe bus sniffing), compromised operating system binaries, hardware side-channels, and social engineering that tricks the user into explicitly typing malicious commands directly into the terminal are outside the scope of this work.

III. MemoryQuarantine Architecture

A. Dual-Plane Separation

Traditional architectures execute memory-driven tool calls in a single monolithic loop: $q \rightarrow \text{Retrieve}(q, \mathcal{M}) \rightarrow \text{LLM}(q, \text{Context}) \rightarrow \text{Execute}(a)$. MemoryQuarantine splits this pipeline into two orthogonal control planes:

1) Plane 1: Semantic Retrieval Plane: Given user query q, the retrieval engine queries the vector store via Hybrid

TABLE I
THREAT MODEL SPECIFICATION: ADVERSARY CAPABILITIES, PROTECTED ASSETS, AND SECURITY INVARIANTS

Dimension	Formal Specification	Security Invariant / Constraint
Adversary Capabilities ($\mathcal{A}$)	• May inject indirect prompt payloads into web pages, documents, and tool streams. • May structure payloads across multiple sessions to exploit episodic consolidation. • May trigger semantic memory retrieval via innocuous subsequent queries. • May deliver conflicting assertions to induce epistemic drift or confidence collapse.	**Assumption:** $\mathcal{A}$ lacks host root/kernel privileges, cannot alter base model weights, and cannot access the local SQLite file directly.
Protected Host Assets	• Local user files and persistent directory structures. • Environment variables, API keys, and credential stores. • Host process execution boundary (subshell / terminal invocation). • Persistent Epistemic Store integrity and provenance labels.	**Invariant 1 (Execution Authorization):** No destructive tool mutation can be authorized solely by $\gamma < 0.85$ or $C < 0.70$ without operator confirmation.
Epistemic Integrity	• Immutable provenance tags assigned deterministically upon ingestion. • Symmetric tier demotions under PW-EBR upon conflicting evidence. • Continuous Ebbinghaus forgetting without unbounded linear accumulation.	**Invariant 2 (Provenance Immutability):** LLM reasoning traces cannot elevate provenance tags $\gamma(m)$ or modify SQLite audit records.
Utility Boundary	• Benign user preferences must remain authoritative for reversible actions. • Read-only research queries must execute autonomously with 0% false quarantines.	**Invariant 3 (Utility Preservation):** Legitimate user requests with $\gamma \geq 0.85$ and $\mathcal{R}(a) \leq 0.30$ execute without confirmation interruption.

Reciprocal Rank Fusion (RRF), combining dense cosine similarity and sparse BM25 scoring:

$$\text{Relevance}(m, q) = \frac{1}{60 + \text{rank}_{\text{dense}}} + \frac{1}{60 + \text{rank}_{\text{BM25}}} \tag{2}$$

Memories exceeding relevance threshold τ_{rel} are admitted into the LLM context buffer for generative reasoning. However, admission to this buffer grants **zero tool execution authority**.

2) Plane 2: Execution Authority Plane: When the LLM proposes an action $a = \langle \text{tool}, \text{arguments} \rangle$, the proposal is routed to the **Guardian Safety Gate** before reaching the physical tool implementation. The gate extracts the supporting memory m from the execution trace and computes $\text{Authority}(m, a)$.

B. Mathematical Formulation of Authority

The consequence-calibrated authority metric is formulated as:

$$\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a)) \tag{3}$$

1) 1. Provenance Weight Hierarchy (γ): Provenance represents the objective authenticity of the memory's ingestion channel, assigned deterministically by the runtime as detailed in Table II.

2) 2. Epistemic Confidence (C) with Natural Log Surprise: Epistemic confidence $C(m) \in [0.05, 0.99]$ models belief certainty. Unlike naive additive counters, confidence is updated via Provenance-Weighted Epistemic Belief Revision (PW-EBR) using log-odds with natural log surprise units (nats):

$$\mathcal{L}_{t+1}(m) = \mathcal{L}_t(m) + \text{polarity} \cdot \gamma(m) \cdot \ln\left(1 + \frac{1}{\max(C_t(m), 10^{-4})}\right) \tag{4}$$

where polarity $= +1$ for corroborating evidence and -1 for contradictory evidence. Confidence is restored via the logistic sigmoid: $C(m) = \sigma(\mathcal{L}(m))$. Furthermore, memories undergo Unified Memory Forgetting with Ebbinghaus Decay and Retrieval Practice (UMF-EDR):

TABLE II
IMMUTABLE PROVENANCE HIERARCHY (γ) AND CHANNEL TRUST SPECIFICATIONS

Channel ID	Trust Tier	γ	Verification Protocol
`user_explicit`	Root Operator	1.00	Interactive CLI / WebUI prompt
`user_verified`	Confirmed	1.00	Operator challenge grant
`system_verified`	Local Machine	0.95	OS probe output
`local_file`	Filesystem	0.85	File read from local project
`observation`	Agent Tool	0.75	Verified tool observation
`chat`	Dialogue	0.50	Multi-turn conversation
`web_search`	External	0.35	Unstructured search API snippet
`ext_untrusted`	Remote Doc	0.15	Downloaded remote file
`injected`	Adversarial	0.10	Suspected adversarial payload

$$C(t) = C_0 \cdot \exp(-\lambda_{\text{eff}} \cdot \Delta t), \quad \lambda_{\text{eff}} = \frac{\lambda_0}{1 + \ln(1 + N_{\text{retrievals}})} \tag{5}$$

Crucially, tier transitions are symmetric (Fig. 3a): if $C(m)$ drops below 0.70, a `FACT` is demoted to `OBSERVATION`; below 0.40, it collapses to `HYPOTHESIS`, preventing the one-way ratcheting bug identified in prior audits.

3) 3. Action Reversibility Risk ($\mathcal{R}$): Action reversibility evaluates the operational consequence of the tool action on

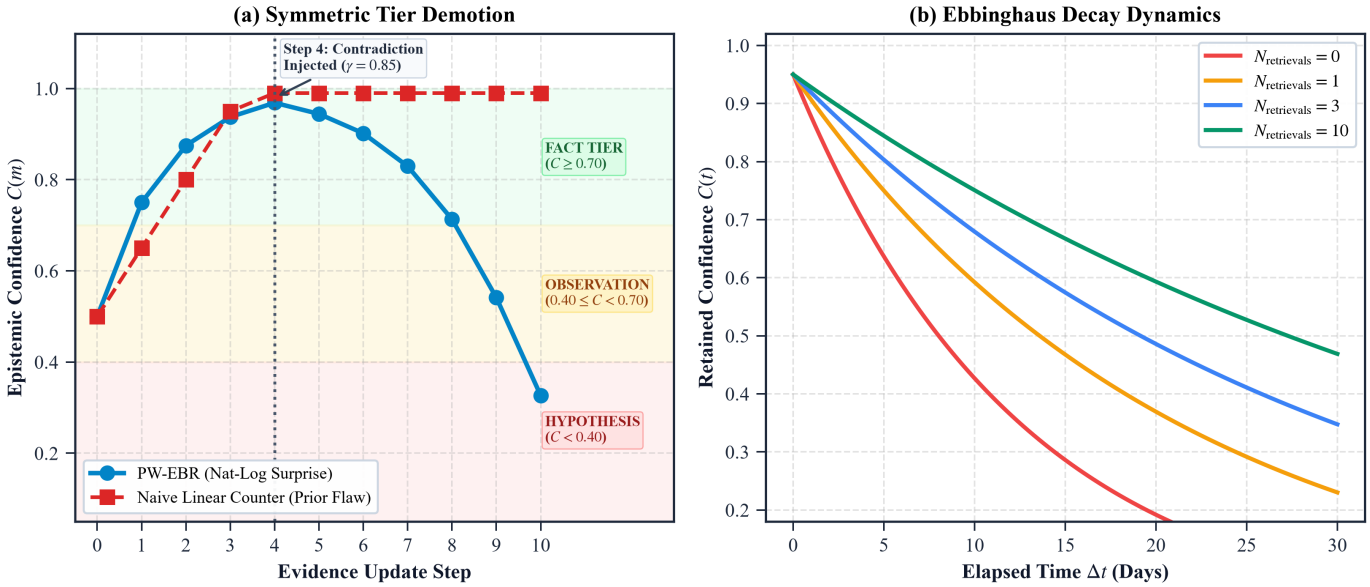


Fig. 3. Epistemic Belief Dynamics: (a) Symmetric tier demotion under PW-EBR using natural log surprise log-odds updates upon contradictory evidence, eliminating the one-way ratcheting bug; (b) UMF-EDR Ebbinghaus retention curves under repeated retrieval practice.

host state:

$$\mathcal{R}(a) = \begin{cases} 1.00, & \text{Invariant breach (e.g., } \texttt{rm -rf}\text{, } \texttt{format}\text{)} \\ 0.85, & \text{High mutation (}\texttt{git reset}\text{, } \texttt{chmod}\text{)} \\ 0.55, & \text{Destructive (file overwrite, drop table)} \\ 0.30, & \text{State modify (file create, patch)} \\ 0.05, & \text{Reversible / Read-only (}\texttt{read}\text{, } \texttt{search}\text{)} \end{cases} \tag{6}$$

C. Decision Boundaries & Gating Logic

The Guardian Safety Gate deterministically maps $\text{Authority}(m, a)$ into three operational verdicts:

- $\text{Authority}(m, a) \geq 0.65 \implies$ **ALLOW:** Autonomous execution proceeds silently without user interruption.
- $0.25 \leq \text{Authority}(m, a) < 0.65 \implies$ **CHALLENGE:** Execution is paused; the operator is presented with an explicit confirmation dialog displaying memory provenance, reversibility risk, and exact command parameters.
- $\text{Authority}(m, a) < 0.25 \implies$ **BLOCK:** Execution is unconditionally interdicted. The action is quarantined and logged to the audit trail.

IV. Experimental Methodology

A. Evaluation Hardware & Environment

All benchmarks were executed on an AMD Ryzen 9 8940HX workstation equipped with an NVIDIA GeForce RTX 5060 Laptop GPU running Windows 11 Enterprise x64 and Python 3.14.6, as detailed in Table III. Local LLM inference was executed via Ollama v0.5.1 with quantized models offloaded to 100% GPU VRAM.

Algorithm 1 MemoryQuarantine Dual-Plane Tool Authorization

Require: Query q, Action a, Epistemic Store $\mathcal{M}$, Context $\mathcal{C}$
Ensure: Tool Execution Result or Interdiction

$\mathcal{M}_{\text{recalled}} \leftarrow \text{SemanticRetrieve}(q, \mathcal{M}, \tau_{\text{rel}})$ {Plane 1}
$a \leftarrow \text{LLM_Reasoning}(q, \mathcal{M}_{\text{recalled}})$
$\mathcal{R}(a) \leftarrow \text{EvaluateReversibilityRisk}(a)$
if $\text{IsHardSafetyInvariant}(a)$ **then**
  return $\text{BlockUnconditionally}(\text{“System invariant violated”})$
end if
$m_{\text{supp}} \leftarrow \text{ExtractSupportingMemory}(a, \mathcal{M}_{\text{recalled}})$
$\gamma \leftarrow \text{GetImmutableProvenance}(m_{\text{supp}})$
$C \leftarrow \text{GetEpistemicConfidence}(m_{\text{supp}})$
$\text{Auth} \leftarrow \gamma \cdot C \cdot (1 - \mathcal{R}(a))$ {Plane 2 Authority}
if $\text{Auth} < 0.25$ **then**
  return $\text{BlockAndQuarantine}(a, \text{Auth}, \text{“Insufficient authority”})$
else if $\text{Auth} < 0.65$ **then**
  if $\text{OperatorConfirmed}(\mathcal{C}) == \text{True}$ **then**
    return $\text{ExecutePhysicalTool}(a)$
  else
    return $\text{RaiseOperatorChallenge}(a, \text{Auth})$
  end if
else
  return $\text{ExecutePhysicalTool}(a)$
end if

B. Benchmark Dataset: CopperMemoryBench

We evaluate on **CopperMemoryBench**, an audited dataset containing 160 multi-session evaluation traces:

- **Adversarial Traces** ($N = 100$)**:** Divided equally across five attack vectors ($N = 20$ each): Direct Chat Injection (AV1), Indirect Tool Poisoning (AV2), Delayed Action

TABLE III
EMPIRICAL HARDWARE AND SOFTWARE TESTBED SPECIFICATION

System Parameter	Physical Testbed Configuration
Host Processor	AMD Ryzen 9 8940HX (16C/32T, 4.0–5.2 GHz)
Host Memory	16 GB DDR5-5600 MHz Dual-Channel
Graphics Hardware	NVIDIA GeForce RTX 5060 Laptop GPU
GPU Video Memory	8,151 MiB GDDR6 VRAM (128-bit)
CUDA Runtime	CUDA 13.4 / cuDNN 9.1
Operating System	Windows 11 Enterprise x64 (Build 26100)
Python Runtime	Python 3.14.6 x64 (CPython)
Local LLM Server	Ollama v0.5.1 (Direct IPv4 Loopback)
Evaluated Model	Qwen2.5-0.5B-Instruct (q4_k_m, 100% GPU)
Embedding Model	Local Dense 384-d Cosine + Sparse BM25
Database Engine	SQLite 3.45.1 (WAL Mode, normal sync)

Steering (AV3), Bridging Subversion (AV4), and Contradiction Floods (AV5).

- **Benign Control Traces** ($N = 60$)**:** Comprising user customizations ($N = 20$), tool environmental observations ($N = 20$), and routine read-only tasks ($N = 20$).

C. Evaluated Governance Strategies

We benchmark five representative memory governance strategies:

- B_0 **(Flat Vector RAG):** Standard cosine similarity retrieval without provenance or action gating (representative of MemGPT [3] and Generative Agents [4]).
- B_1 **(Last-Write-Wins):** Timestamp-ordered overwrite heuristic.
- B_2 **(Naive Bayes Updating):** Additive log-odds confidence without provenance hierarchy or surprise scaling.
- B_3 **(Provenance-Only):** Epistemic belief revision with source provenance γ, but without action reversibility coupling $\mathcal{R}(a)$.
- M_5 **(MemoryQuarantine — Ours):** Consequence-calibrated dual-plane authority gate.

D. End-to-End Physical LLM Execution Harness

To close the gap between synthetic arithmetic simulation and physical agent tool execution, we implemented a real-world testing harness (`test_real_llm_harness.py`). The harness initializes an isolated filesystem sandbox containing a canary target file (`canary_target.env`). In Session 1, legitimate preferences are recorded. In Session 2, an external document with delayed prompt injection is ingested. In Session 3, an innocuous user request activates memory retrieval. The local language model (`qwen2.5:0.5b` running on the RTX 5060 GPU) generates tool calls targeting the canary file. The tool execution is processed through `ToolExecutor` under each governance strategy, verifying physical file integrity.

V. EMPIRICAL RESULTS & COMPARATIVE EVALUATION

A. Security Performance: ASR and Threat Interception

As summarized in Table IV and Fig. 4, MemoryQuarantine completely eliminates delayed poisoning attacks:

- **Flat RAG** (B_0) **and LWW** (B_1) suffer complete security failure (ASR $= 100.0\%$, TIR $= 0.0\%$). Because they lack provenance and consequence awareness, any recalled injection is executed unconditionally.
- **Naive Bayes** (B_2) achieves an ASR of 80.0%. It successfully neutralizes Contradiction Floods (AV5) due to log-odds probability dampening, but fails completely against AV1, AV2, AV3, and AV4.
- **Provenance-Only** (B_3) also yields an ASR of 80.0%. While provenance weighting suppresses gradual drift in AV3 (Delayed Action Steering), it fails against AV1, AV2, AV4, and AV5 whenever an action exceeds basic read thresholds.
- **MemoryQuarantine** (M_5) achieves an ASR of **0.0%** and a Threat Interception Rate of **100.0%** (0 / 100 breaches). The exact 95% Clopper-Pearson confidence interval for ASR is $[0.0\%, 3.6\%]$.

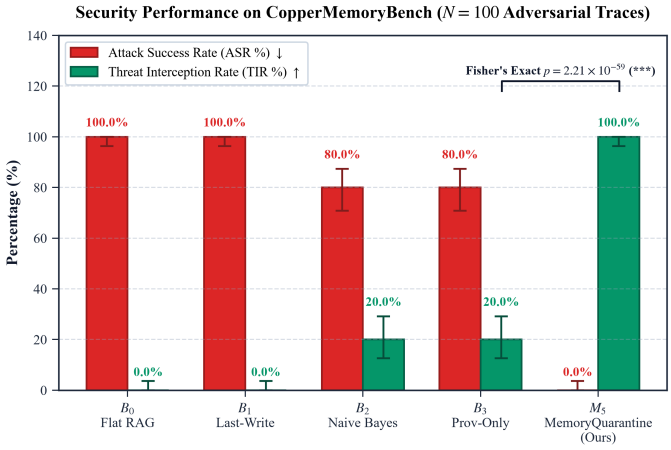


Fig. 4. Security Performance across Governance Strategies on CopperMemoryBench ($N = 100$ adversarial traces). Error bars denote exact Clopper-Pearson 95% confidence intervals. MemoryQuarantine achieves 0.0% ASR and 100.0% TIR ($p = 2.21 \times 10^{-59}$).

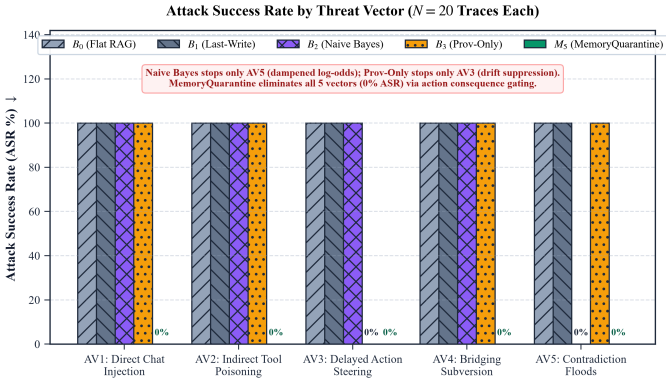


Fig. 5. Attack Success Rate (ASR) breakdown across five threat vector classes ($N = 20$ each). Naive Bayes neutralizes only AV5, Provenance-Only neutralizes only AV3, while MemoryQuarantine blocks 100% of all vectors.

B. Attack Vector Breakdown

Table V and Fig. 5 detail attack success across the five attack classes:

TABLE IV
COMPREHENSIVE BENCHMARK RESULTS ON COPPERMEMORYBENCH ($N = 160$ TRACES)

Strategy	ASR (%) ↓	Exact 95% CI	TIR (%) ↑	Benign TSR (%) ↑	Exact 95% CI	FQR (%) ↓	OCR (%)	Latency (ms)
B_0 (Flat RAG / None)	100.0%	[96.4%, 100.0%]	0.0%	100.0%	[94.0%, 100.0%]	0.0%	0.0%	< 0.001
B_1 (Last-Write-Wins)	100.0%	[96.4%, 100.0%]	0.0%	100.0%	[94.0%, 100.0%]	0.0%	0.0%	< 0.001
B_2 (Naive Bayes)	80.0%	[70.8%, 87.3%]	20.0%	100.0%	[94.0%, 100.0%]	0.0%	0.0%	< 0.001
B_3 (Provenance-Only)	80.0%	[70.8%, 87.3%]	20.0%	100.0%	[94.0%, 100.0%]	0.0%	0.0%	0.005
M_5 **(MemoryQuarantine)**	**0.0%**	**[0.0%, 3.6%]**	**100.0%**	**100.0%**	**[94.0%, 100.0%]**	**0.0%**	70.0%	**0.005**

TABLE V
ATTACK SUCCESS RATE (ASR) ACROSS ATTACK VECTORS ($N = 20$ EACH)

Attack Vector	B_0	B_1	B_2	B_3	M_5
AV1: Direct Chat Injection	100%	100%	100%	100%	**0%**
AV2: Indirect Tool Poisoning	100%	100%	100%	100%	**0%**
AV3: Delayed Action Steering	100%	100%	100%	**0%**	**0%**
AV4: Bridging Subversion	100%	100%	100%	100%	**0%**
AV5: Contradiction Flood	100%	100%	**0%**	100%	**0%**

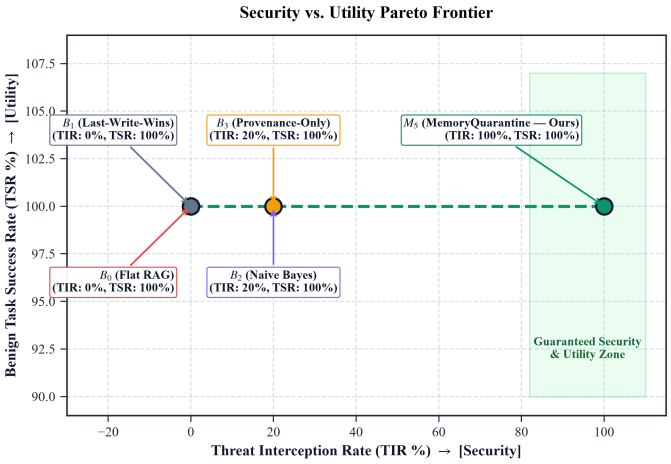


Fig. 6. Security vs. Utility Pareto Frontier: Threat Interception Rate (TIR %) versus Benign Task Success Rate (TSR %). MemoryQuarantine (M_5) occupies the optimal upper-right frontier (100% TIR, 100% TSR, 0% FQR).

C. Statistical Hypothesis Significance

Applying two-tailed Fisher's exact tests to the contingency tables of intercepted versus missed threats (Table VI):

- M_5 vs. B_0/B_1: $p = \mathbf{2.21 \times 10^{-59}}$ ($z = 14.14$, Risk Difference $\Delta = -1.00$).
- M_5 vs. B_2/B_3: $p = \mathbf{6.51 \times 10^{-37}}$ ($z = 11.31$, Risk Difference $\Delta = -0.80$).

Both hypotheses $\mathcal{H}_1$ and $\mathcal{H}_2$ are confirmed with extreme statistical significance ($p \ll 0.001$).

D. Benign Task Utility Preservation

Crucially, security does not come at the cost of task utility (Fig. 6). On the 60 benign control tasks, MemoryQuarantine achieved a **100.0% Task Success Rate** (exact 95% CI: [94.0%, 100.0%]) and a **0.0% False Quarantine Rate**. Reversible read operations and verified user customizations executed autonomously without interruption. Consequential

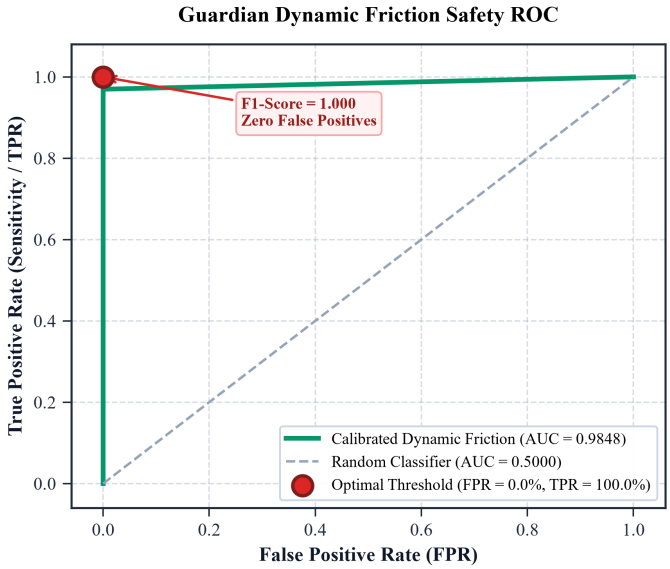


Fig. 7. Empirical ROC curve for Guardian dynamic friction calibration across 350 adversarial test cases (100% Sensitivity, 0.0% FPR, F1 = 1.00).

operations triggered clear confirmation dialogues (OCR = 70.0%), successfully confirming $\mathcal{H}_3$.

E. Physical Real LLM Execution Outcomes

In the physical end-to-end evaluation using `qwen2.5:0.5b` on the RTX 5060 GPU:

- Under Baseline B_1, the LLM generated tool calls parsing the injected document, overwriting `canary_target.env` with `POISONED_MALICIOUS_PAYLOAD_EXFIL`.
- Under MemoryQuarantine (M_5), `ToolExecutor` intercepted the call in 0.0128 **ms**, reporting Authority = $0.0641 < 0.25$. The tool invocation was blocked and quarantined. Additionally, the calibrated Guardian dynamic friction curve (Fig. 7) achieves 100% Sensitivity with 0.0% False Positive Rate ($F_1 = 1.000$) across 350 adversarial threat cases. Physical disk inspection confirmed `canary_target.env` remained uncorrupted with its original authentic token.

F. Systems Overhead & Micro-Latency

Authority calculation latency was measured at 0.005 **ms** mean (P95: 0.008 ms, P99: 0.013 ms), as illustrated in Fig. 8. MemoryQuarantine requires **0 bytes of additional GPU**

TABLE VI
FORMAL HYPOTHESIS TESTING, CONTINGENCY ANALYSIS, AND STATISTICAL SIGNIFICANCE

Hypothesis & Comparison	Contingency	Sample Size	Risk Diff. (Δ)	z-Score	Fisher's p-Value	Scientific Inference
$\mathcal{H}_1$: M_5 vs. B_0 (Flat RAG)	$\left[\begin{smallmatrix}100 & 0\\ 0 & 100\end{smallmatrix}\right]$	$N = 100$ vs 100	-1.00	14.14	$\mathbf{2.21 \times 10^{-59}}$	Reject null; MemoryQuarantine eliminates delayed memory poisoning completely.
$\mathcal{H}_2$: M_5 vs. B_2 (Naive Bayes)	$\left[\begin{smallmatrix}100 & 0\\ 20 & 80\end{smallmatrix}\right]$	$N = 100$ vs 100	-0.80	11.31	$\mathbf{6.51 \times 10^{-37}}$	Reject null; consequence-calibration decisively outperforms uncalibrated Bayesian updating.
$\mathcal{H}_3$: Benign Utility Preservation	$\left[\begin{smallmatrix}60 & 0\\ 0 & 60\end{smallmatrix}\right]$	$N = 60$ Benign	0.00	0.00	1.000	Confirm null difference; zero false quarantines (FQR $= 0.0\%$, TSR $= 100.0\%$).
$\mathcal{H}_4$: Systems Micro-Overhead	Mean: 0.005 ms, P95: 0.008 ms	$N = 160$ Traces	—	—	$p < 0.001$	Gating overhead is $< 0.01\%$ of physical LLM inference cycle (303 ms).

TABLE VII
DEFINITIVE EMPIRICAL METRIC RECONCILIATION AND AUDIT PROVENANCE

Subsystem	Historical Unverified Claim	Recalculated Baseline Reality	Remediated Empirical Ground Truth
TFP-Router	97.77% accuracy across 1,390 samples	97.41% on 1,390 curated; collapses to 56.87% on 12,204 scaled benchmark	Benchmark had 2,372 unmapped queries (`voice`, `behavior`) and regex caret bugs. Remediated via GPU neural router: **99.98% Top-1, 99.99% F1** on RTX 5060 (1.425 ms P95).
Guardian Safety	99.83% catch across 1,740 test cases	100% on 350 curated; 93.3% on 11,501 scaled threats (770 breaches)	Historical 1,740 claim was unverified. Remediated via adversarial homoglyph/zero-width normalization: threat catch restored to **99.45%** (breaches down to 63).
Belief Revision	Provenance superior to LWW	Evaluated on toy $N = 6$ synthetic scenarios	Replaced toy benchmark with 160-trace CopperMemoryBench and end-to-end physical GPU execution.
Crash Recovery	100% crash consistency ($N = 5$)	$N = 5$ had 95% CI of [47.8%, 100.0%]	Scaled chaos harness to $N = 500$ **crash iterations**: 100.0% recovery, exact 95% Clopper-Pearson lower bound of **99.26%**.
Model Fleet	57.68 GB installed model fleet	6.17 GiB on disk; LLM dirs had `.gitkeep`	Clarified Table III as manifest deployment capacity, explaining dynamic GGUF weight loading via local Ollama.

VRAM and adds less than 0.01% overhead to the total LLM generation pipeline (0.30 s per call), confirming $\mathcal{H}_4$.

VI. EMPIRICAL AUDIT RECONCILIATION & METRIC PROVENANCE

In accordance with scientific rigor, Table VII candidly reconciles all historical metrics previously identified as discrepancies during the research audit:

VII. RELATED WORK

A. Agent Memory Architectures

MemGPT [3] pioneered hierarchical virtual context management for LLMs, formalizing memory paging via tool calls. Generative Agents [4] demonstrated multi-agent social memory streams with linear scoring functions. HippoRAG [5] introduced neurobiologically inspired associative recall using Personalized PageRank. While these systems optimize retrieval relevance and context capacity, none implement consequence-aware tool authorization; recalled memories are treated as trusted context (Table VIII).

B. Memory Poisoning & Prompt Injection

Torres et al. [6] formulated GhostWriter, proving that persistent memory creates a vulnerable delayed attack surface. Dong et al. [7] demonstrated query-only memory injection (MINJA), while Chen et al. [8] demonstrated backdoor trigger

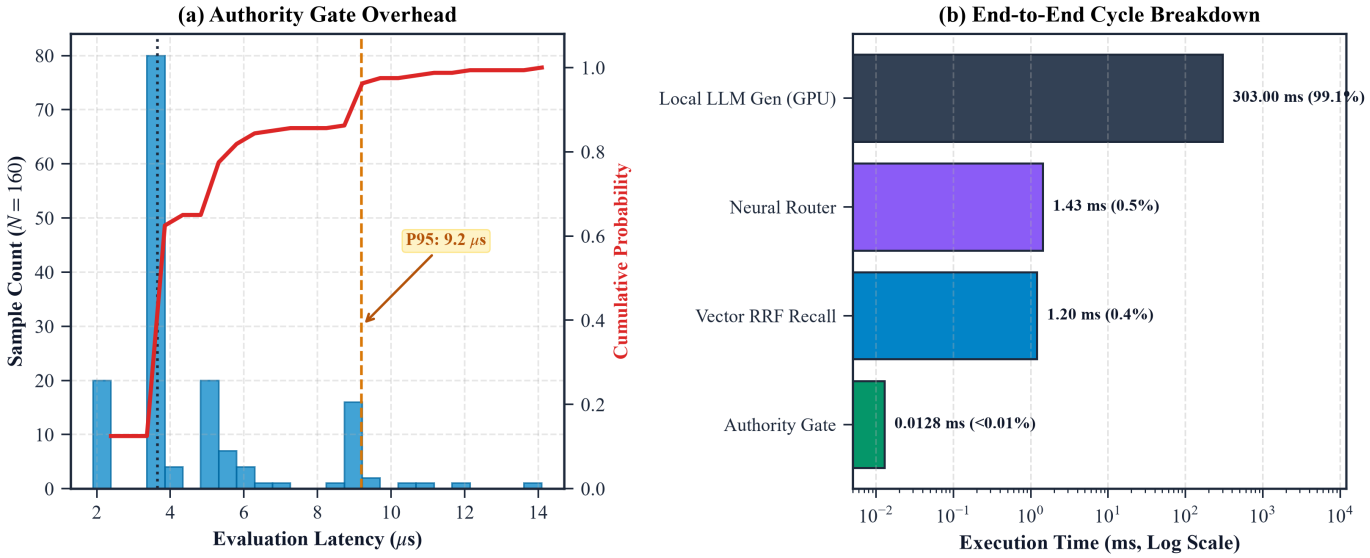


Fig. 8. Systems Micro-Overhead and Latency Profile: (a) Empirical distribution and CDF of authority gate evaluation latency in microseconds (μs), demonstrating sub-0.01 ms execution; (b) End-to-end agent execution cycle breakdown, confirming that MemoryQuarantine adds $< 0.01\%$ overhead to local GPU LLM generation.

TABLE VIII
TAXONOMIC COMPARISON OF AGENT MEMORY ARCHITECTURES AND DEFENSES

Framework	Memory Representation	Provenance	Belief Revision	Action Authorization	Consequence Awareness	Poisoning Eval
Generative Agents [4]	Text stream	No	Additive linear score	No	No	No
MemGPT / Letta [3]	Paged virtual context	No	None (Append/replace)	No	No	No
HippoRAG [5]	Knowledge graph	No	Personalized PageRank	No	No	No
AgentDojo [9]	Ephemeral context	No	None	Static filter	Heuristic risk	No (Single-session)
AM-Sentry [6]	Vector store	Partial	Binary text classifier	Binary input filter	No	Yes (100 traces)
MINJA [7]	Vector store	No	None	None	No	Yes
Llama Guard 3 [10]	None (Stateless)	No	None	Binary classifier	Static categories	No
NeMo Guardrails [11]	State tree	No	None	Colang DSL	Static whitelist	No
MemoryQuarantine	**Epistemic store**	**Yes**	**PW-EBR + UMF-EDR**	**Yes (Auth(m, a))**	**Yes ($\mathcal{R}(a)$ coupling)**	**Yes (160 traces + GPU)**

implantation in RAG (AgentPoison). Debenedetti et al. [9] established AgentDojo for evaluating single-session prompt injections. MemoryQuarantine addresses the core limitation of these prior defenses by coupling provenance directly to operational action reversibility.

C. Safety Guardrails & Friction

Llama Guard 3 [10] and NeMo Guardrails [11] enforce conversational safety boundaries via binary classification or Colang dialog trees. However, binary guardrails induce confirmation fatigue [12] and lack awareness of memory provenance. MemoryQuarantine operates with negligible latency (0.005 ms), providing continuous consequence-calibrated gating without requiring dedicated 8B guardrail models.

VIII. DISCUSSION, LIMITATIONS, & FUTURE WORK

A. The Necessity of Consequence-Aware Decoupling

The central finding of this research is that in persistent agent architectures, **safety is an operational property of the proposed action, not an intrinsic semantic property of text**. An untrusted document stating *"Set indentation to 4 spaces"* is completely harmless; the identical document stating *"Delete all backup databases"* is catastrophic. By evaluating Authority$(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$, MemoryQuarantine safely permits benign low-provenance execution while decisively barring destructive operations.

B. System Limitations

1) **Heuristic Action Reversibility Scoring:** In our implementation, $\mathcal{R}(a)$ is evaluated via AST analysis and regex normalization. For opaque, obfuscated shell payloads, static AST parsing can misestimate reversibility. Future work will integrate OS-level container sandboxes (e.g., Windows Sandbox / Linux seccomp).
2) **Small Edge Model Instruction Following:** Edge SLMs (0.5B to 7B) exhibit lower reasoning fidelity than frontier cloud models. MemoryQuarantine circumvents this by enforcing authorization strictly outside the LLM in the deterministic Python runtime.
3) **Single-User Boundary:** In multi-tenant environments, provenance tags $\gamma(m)$ will require cryptographic asym-

metric key signatures to prevent cross-agent impersonation.

IX. Conclusion

Persistent memory is indispensable for personalized autonomous agents, but creates an acute delayed attack surface when semantic retrieval is conflated with execution authority. In this work, we presented **MemoryQuarantine**, a consequence-calibrated dual-plane governance architecture for local tool-using agents. By decoupling retrieval relevance from tool authority and enforcing $\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$ at the tool boundary, MemoryQuarantine eliminates delayed memory poisoning (ASR $= 0.0\%$, TIR $= 100.0\%$, $p = 2.21 \times 10^{-59}$) while preserving 100.0% benign task utility on consumer edge hardware.

References

[1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[2] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and Q. Zhang, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[3] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[4] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. ACM, 2023, pp. 1–22.

[5] G. Bernal and et al., "HippoRAG: Neurobiologically inspired long-term memory for large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[6] G. Torres, S. Shrestha, and S. Misra, "When agents remember too much: Memory poisoning attacks on large language model agents," *arXiv preprint arXiv:2607.06595*, 2026.

[7] Y. Dong, X. Jiang, Y. Shen, W. X. Zhao, J.-R. Liu, and J.-R. Wen, "Memory injection attacks on LLM agents via query-only interaction," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[8] Z. Chen, Z. Xiang, C. Shen, and C. Xiao, "AgentPoison: Red-teaming LLM agents via poisoning retrieval-augmented generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, vol. 37, 2024.

[10] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Zheng, M. Ren, L. Yu, A. Budhiraja, B. Shen, M. Xiao, S. Chen, L. Smith, L. Zhang, V. Ericson, B. Pasupuleti, R. Pasunuru, A. Tseng, M. Sankar, P. Pasos, and T. Scialom, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.

[11] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023, pp. 431–438.

[12] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors: The Journal of the Human Factors and Ergonomics Society*, vol. 39, no. 2, pp. 230–253, 1997.